Amazon DynamoDB Basics

Understand Amazon DynamoDB's core concepts such as tables, items, attributes, and primary keys. Explore how to model data with partition and composite keys, serialize Python objects for storage, retrieve data, and optimize read performance using DynamoDB Accelerator (DAX). This lesson prepares you to design scalable, resilient cloud-native applications using DynamoDB for the AWS Developer Associate exam.

DynamoDB is Amazon's fully managed NoSQL database designed for millisecond performance at scale. It's a perfect fit for applications requiring high availability, flexible schemas, and seamless scaling.

DynamoDB stores data in tables, which contain items, and each item is composed of attributes. A table functions similarly to a relational database table, storing structured information related to a specific entity, such as users, orders, or products. Each item in a DynamoDB table is a unique record, much like a row in a relational table, although the schema may vary across items. Attributes represent an item's data fields, similar to columns in other databases.

DynamoDB is built on persistent storage systems. This means that once d written to DynamoDB, it remains available even if a hardware failure occurs, an

Ask

instance reboots, or other system maintenance happens. This durability is a key part of DynamoDB's managed service guarantees.

In contrast, some services use ephemeral storage, which is temporary and lost when the underlying resource (like an EC2 instance) is stopped or terminated. While using DynamoDB tables:

- The data is replicated across multiple Availability Zones (AZs) within a Region to ensure durability and availability.
- We do not need to worry about manually backing up your tables for basic durability (although backup and restore features exist for compliance and recovery purposes).

Understanding the difference helps developers design resilient, scalable cloud-native applications without worrying about underlying hardware.

Primary key structure

Every table must specify a primary key, which determines how data is distributed and queried. The primary key can be either a simple partition key or a composite key:

- **Partition key:** When using a partition key (also called a hash key), DynamoDB uses the key's value to determine the partition, or physical storage node, on which to store the item. This mechanism enables high availability and scalability. To optimize performance, partition keys should have high cardinality and even distribution across items.
- **Composite primary key:** It combines a partition key with a **sort key,** also known as a range key. This enables multiple items to share the same partition key while being uniquely distinguished and sorted by the sort key. Composite keys support ordered queries within a partition, making th ideal for modeling one-to-many relationships.

Ask

How data is stored in DynamoDB partitions

Schema definitions in DynamoDB

DynamoDB is schema-less for attributes. This means we don't need to predefine all the columns as we do in relational databases. Instead, only the attributes that form the primary key (partition and sort keys) and any secondary indexes must be defined at table creation.

This flexible attribute design allows us to evolve item structure over time, enabling us to store application objects with varying shapes. For example, the DynamoDB schema hierarchy illustration given below shows two items stored in the DynamoDB table.

Ask

Schema hierarchy in DynamoDB table

Notice that each item corresponds to a different event type and has different attributes. However, DynamoDB allows us to store both items in the same table.

However, this freedom comes with a need for discipline: consistency in attribute naming and typing is critical to avoid complex client-side logic or query failures. In simple terms, schema does matter in the definition of keys and indexes. Key and indexed attributes must follow consistent typing and usage, since they directly influence query behavior and storage layout.

Storing and retrieving objects in DynamoDB

When working with application data, it's important to understand that DynamoDB expects information to be stored in simple, structured types: strings, numbers, booleans, lists, and maps. DynamoDB is a **typed database** at the wire protocol level. It needs to know exactly:

- Whether a value is a string, number, boolean, list, or map.
- How to store and index that value internally.

Ask

Real world example in Python: Social media posts

We'd better understand these concepts with a real-world example. Imagine building a simple social media application where users can create multiple posts. Each post needs to be stored in DynamoDB, and we need a smart way to organize, retrieve, and scale our data.

Let's walk through it together.

Step 1: Create a DynamoDB table

The primary key is at the heart of every DynamoDB table, and in our `UserPost` database, we have a choice: simple or composite.

Since a single user will create many posts, a simple partition key won't be enough — we need to model a one-to-many relationship. Thus, we'll use a composite primary key.

- **Partition Key:** `UserId`
- **Sort Key:** `PostTimestamp`

UserPosts table in DynamoDB

Ask

Here, the `UserId` ensures data distribution across partitions and `PostTimestamp` enables quick retrieval of recent posts, sorted by creation time. We can run

queries like “Show me the last 10 posts by this user” efficiently.

If we had only used `UserId` as a simple partition key, we wouldn’t be able to differentiate between multiple posts from the same user without scanning or adding complexity elsewhere.

This means all posts from the same user will be grouped together, but uniquely sorted by the time they were created.

Here’s the Python code to create the table:

main.py Python

```
import boto3

# Create DynamoDB client
dynamodb = boto3.client(
    'dynamodb',
    region_name="us-east-1"
)

# Create the UserPosts table
response = dynamodb.create_table(
    TableName='UserPosts',
    AttributeDefinitions=[
        {'AttributeName': 'UserId', 'AttributeType': 'S'}
```

Python code to create UserPosts table

Let’s understand the code:

- **Lines 11–15:** We define the `UserPosts` table attributes. This sets the `UserId` as a string and `PostTimestamp` as a number.
- **Lines 16–19:** We define the schema of `UserPosts` table. It sets `UserI` the partition key and `PostTimestamp` as the sort key, forming a composite

Ask

primary key. This design enables efficient storage and retrieval of multiple posts per user, ordered by time.

- **Line 20:** We set up on-demand capacity scaling, so you pay only for actual usage.

Step 2: Saving objects to the DynamoDB table

Before saving an object like a social media post to the DynamoDB table, we must serialize it—convert it from a native Python object into a dictionary format that DynamoDB can store. DynamoDB accepts the following types of values:

- Strings (`S`)
- Numbers (`N`)
- Booleans (`BOOL`)
- Lists (`L`)
- Maps (`M`)
- Null values (`NULL`)

Let's serialize this into DynamoDB-compatible data types:

main.py Python

```python
import boto3
import time

# Create DynamoDB client
dynamodb = boto3.client(
    'dynamodb',
    region_name='us-east-1'
)

# Example post object
post = {
    "UserId": "user123",
    "PostTimestamp": int(time.time())  # Current Unix timestamp
```

Ask

Python code to add objects to the UserPosts table

Let's understand the code:

- **Lines 11–17:** We define a simple `post` object. Here, note that `UserId` and `PostTimestamp` are required because they form the primary key.
- **Lines 20–29:** We use the `put_item` API to store the post. Each attribute is serialized to DynamoDB types manually:
  - `S` for strings (`UserId`, `Title`, `Content`, `Status`).
  - `N` for numbers (`PostTimestamp`).

Step 3: Retrieving objects from the DynamoDB table

Similarly, when retrieving data, we deserialize the DynamoDB response back into a Python object. For example, the code below fetches the user's post that we added to the table in the previous section.

main.py C++

```
import boto3
import time

# Create DynamoDB client
dynamodb = boto3.client(
    'dynamodb',
    region_name='us-east-1'
)

# Keys used to fetch the item (must match what was inserted)
user_id = "user123"
post_timestamp = <POST-TIME-STAMP>  # should match the timestamp used
```

Ask

Serialized user post object ready to be stored in DynamoDB table

When fetched, it must be deserialized back into the application's internal representation to continue processing or display.

AWS SDKs (like Boto3) help automate this serialization and deserialization, making development smoother and reducing manual mapping work.

DynamoDB Accelerator (DAX)

DynamoDB Accelerator is a caching solution for DynamoDB tables. It's a fully managed service that enhances DynamoDB's performance by providing a microsecond response time. It is useful for scenarios where fast retrieval of eventually consistent data is required. It saves cost by excluding the need to provision extra resources and brings frequently accessed data into memory for better performance.

DAX is a cluster of cache nodes (one primary and the other read replicas) that reside between the application and DynamoDB. It's a VPC-based resource and always runs in a VPC. When the application generates API requests, they are routed toward the DAX cluster. If the DAX cluster fulfills the request, the data is returned to the application. Otherwise, the request is passed to the DynamoDB tables.

Conclusion

DynamoDB is Amazon's fully managed NoSQL database designed for applications that demand high availability, flexible schemas, and millisecond performance at scale. Data is organized into tables, with each table storing

Ask

uniquely identified items composed of attributes, allowing for flexible and evolving data structures. Every table requires a primary key—either a simple partition key or a composite key with a sort key—which determines how data is partitioned,

Ask